

Wording for P2644R1 Fix for Range-based for Loop

Document#: P2718R0
Date: 2022-11-11
Project: ISO JTC1/SC22/WG21: Programming Language C++
Audience subgroup: Core
Reply-to: Nicolai Josuttis <nico@josuttis.de>
Joshua Berne <jberne4@bloomberg.net>

Introduction

This contains the wording extracted from P2644R1 (Final Fix of Broken Range-based for Loop) and then updated based on guidance from CWG November 10 and 11, 2022.

Proposed Wording

The following is in reference to N4919.

☐ Hide text inserted in-line ☐ Hide text deleted in-line

In subclause 6.7.7 [class.temporary], modify p5:

There are ~~three~~four contexts in which temporaries are destroyed at a different point than the end of the full-expression. ...

In subclause 6.7.7 [class.temporary], add a paragraph after p6

The fourth context is when a temporary object other than a function parameter object is created in the *for-range-initializer* of a range-based for statement. If such a temporary object would otherwise be destroyed at the end of the *for-range-initializer* full-expression, the object persists for the lifetime of the reference initialized by the *for-range-initializer*.

In subclause 6.7.7 [class.temporary], modify p7

The destruction of a temporary whose lifetime is not extended beyond the full-expression in which it was created~~by being bound to a reference~~ is sequenced before the destruction of every temporary which is constructed earlier in the same full-expression. If the lifetime of two or more temporaries with lifetimes extending beyond the full-expressions in which they were created~~to which references are bound~~ ends at the same point, these temporaries are destroyed at that point in the reverse order of the completion of their construction. In addition, the destruction of such~~bound to references~~ temporaries shall take into account the ordering of destruction of objects with static, thread, or automatic storage duration (6.7.5.2, 6.7.5.3, 6.7.5.4); that is, if obj1 is an object with the same storage duration as the temporary and created before the temporary is created the temporary shall be destroyed before obj1 is destroyed; if obj2 is an object with the same storage duration as the temporary and created after the temporary is created the temporary shall be destroyed after obj2 is destroyed.

In subclause 8.6.5 [stmt.ranged] add before Example 1:

[Note: The lifetime of some temporaries in the *for-range-initializer* is extended to cover the entire loop (6.7.7 [class.temporary]). --- end note] [Example:

```
using T = std::list<int>;
const T& f1(const T& t) { return t; }
const T& f2(T t)        { return t; }
T g();
void foo() {
    for (auto e : f1(g())) {} // OK, lifetime of return value of g() extended
    for (auto e : f2(g())) {} // undefined behavior
}

--- end example]
```

Add a new section in Annex C:

Affected subclause: 8.6.5 [stmt.ranged]

Change: The lifetime of temporary objects in the *for-range-initializer* is extended until the end of the loop (6.7.7 [class.temporary]).

Rationale: Improve usability of the range-based for loop.

Effect on original feature: Destructors of some temporary objects are invoked later.

[Example1:

```
void f()
{
    std::vector<int> v = { 42, 17, 13 };
    std::mutex m;
    for (int x : static_cast<void>(std::lock_guard<std::mutex>(m)), v) // lock released in C++ 2020
    {
        std::lock_guard<std::mutex> guard(m); // OK in C++ 2020, now deadlocks
    }
}

-- end example]
```